

Project Concept Note

Document Analyzer — AI-Powered Document Intelligence

Application Name	Document Analyzer
LLM / API Used	Anthropic Claude (claude-sonnet-4-6) via the Messages API, streaming mode
Live Application URL	<PASTE YOUR AWS APP RUNNER / DEPLOYMENT URL HERE>
Tech Stack	FastAPI (Python) backend · HTML/CSS/JS frontend · Docker · AWS App Runner

1. Problem Statement / Objective

Professionals across research, legal, business, and academic settings routinely receive long, dense documents — contracts, reports, articles, policy papers — that are time-consuming to read closely. Manually summarizing, extracting key facts from, or rewriting these documents is slow and inconsistent. Document Analyzer solves this by letting a user upload a PDF or paste text and instantly receive an AI-generated summary, structured extraction, professional rewrite, or critical analysis, with the response streaming in live rather than appearing after a long wait.

2. Target User and Use Case

- Knowledge workers and students who need to quickly digest long reports, articles, or research papers.
- Legal, compliance, and business professionals who need key facts, dates, and obligations extracted from contracts or policies.
- Writers and communicators who want a document rewritten to be clearer and more professional.
- Analysts who need a structured, critical read of a document's purpose, strengths, and gaps.

3. LLM Model and API Used

The backend integrates with Anthropic's Claude API (model: **claude-sonnet-4-6**) using the official Python SDK's streaming Messages endpoint. The API key is stored exclusively as a server-side environment variable / AWS Systems Manager SecureString parameter and is never exposed to the browser or committed to version control.

4. Key Features

- PDF upload (drag-and-drop or file picker) with server-side text extraction, or direct text paste.
- Four analysis modes: Summarize, Extract Key Information, Rewrite & Polish, and Deep Analysis, each backed by a tailored prompt template.
- Optional custom instructions field so users can steer the analysis (e.g. 'focus on financial figures').
- Real-time, token-by-token streaming of the AI response via Server-Sent Events (SSE), rendered as formatted Markdown.
- Responsive, mobile-optimized two-panel interface (input on one side, live output on the other).

- Copy-to-clipboard and clear/reset controls for the generated output.
- Backend guardrails: file size limits, input length truncation, encrypted/scanned-PDF handling, and structured error messages.

5. Expected User Experience and Outcomes

A user lands on a clean, single-screen interface, uploads a PDF or pastes text, chooses what they need done, and clicks Analyze. Within roughly a second the response begins streaming into the results panel word-by-word, similar to a modern AI chat experience, so the app feels responsive even though the full generation may take several seconds. The outcome is a document task — reading a 10-page report, pulling out obligations from a contract, or polishing a draft — that used to take many minutes reduced to under a minute, with a result the user can copy out directly.

6. Architecture Overview

Browser (HTML/CSS/JS) → FastAPI backend (Python, containerized) → Anthropic Claude API (streaming). The FastAPI service both serves the static frontend and exposes two POST endpoints (`/api/analyze/pdf` and `/api/analyze/text`) that stream Server-Sent Events back to the browser. The entire stack is packaged into one Docker image and deployed to AWS App Runner behind a public HTTPS endpoint, with the Claude API key injected at runtime as a secret.